1. CONFORMACIÓN DEL EQUIPO SCRUM Y VALIDACIÓN DE ROLES

Miembro	Rol
Bianca	Product Owner (PO)
Franco	Scrum Master (SM)
Benja, Alison, Matias, Leandro	Developer

2. SPRINT PLANNING: PARÁMETROS DEL SPRINT E HISTORIAS

Parámetros	Definición para Sprint #2
Sprint Goal	Desarrollo de la construcción de una clase y luego agregarla al calendario semanal.
Time-box	7 días
Capacidad / SP	8 SP
Sprint Backlog	Crear clase, agregar clase y visualizarla en el calendario.
Tablero Kanban	POR HACER (To Do) | EN CURSO (In Prog) | EN REVISIÓN | FINALIZADO (Done)

2.1 Historia de Usuario:

ID y Título: US-45 Crear Clase

Estructura Card (3C): Como administrador del gimnasio, quiero crear una Clase indicando la actividad, el profesor y el horario.

Criterios de aceptación:

Escenario 1: Registrar una clase correctamente

- Dado que el administrador accede al panel de gestión de clases,
- Cuando completa los datos obligatorios de la clase y confirma el registro,
- Entonces el sistema guarda la clase con la información ingresada

Escenario 2: Intentar registrar una clase con datos obligatorios incompletos

- Dado que el administrador se encuentra en el formulario de registro de clases,
- Cuando intenta guardar una clase sin completar alguno de los campos obligatorios,
- Entonces el sistema impide el registro e informa cuáles son los datos que debe completar.

Escenario 3: Visualización de una clase registrada

- Dado que el administrador registró correctamente una clase,
- Cuando un cliente consulta el calendario de actividades,
- Entonces el sistema muestra la clase con la información necesaria para conocer la actividad, el profesor y el horario, y permite acceder a la opción de inscripción según las reglas del sistema.

Escenario 4 : No existen clases repetidas

- Si el administrador intenta registrar una clase correctamente, si esta existe en el sistema o se superpone con otro horario no debería permitirse la creación de la clase

ID y Título: US-21 Agregar Clase

Estructura Card (3C): Como administrador del gimnasio, quiero registrar una clase indicando la actividad, el profesor y el horario, para que los clientes puedan consultar las actividades disponibles e inscribirse.

Criterios de aceptación:

Escenario 1: Registro exitoso de una clase

- Dado que el administrador accede al panel de gestión de clases,
- Cuando completa los datos obligatorios de la actividad, el profesor y el horario, y confirma el registro,
- Entonces el sistema guarda la clase

Escenario 2: Registro con datos incompletos

- Dado que el administrador se encuentra en el formulario de registro,
- Cuando intenta guardar la clase sin completar uno o más campos obligatorios,
- Entonces el sistema impide el registro e informa qué datos faltan.

Escenario 3: Consulta de la clase registrada

- Dado que el administrador registró correctamente una clase,
- Cuando un cliente consulta el calendario de actividades,
- Entonces el sistema muestra la clase registrada y permite acceder a la opción de inscripción.

2.2 Desglose técnico en Sub-tasks:

ID y Título: US-45 Crear Clase

ID	Descripción	Responsable	Est. Hrs.
US-52	Corregir las validaciones: no pueden haber clases en el mismo horario y día.	Lean	
US-54	Quitar todo el hardcodeo que hay en el componente “Schedule”.	Benja	
US-55	Refactorizar el componente “Schedule” usando el Service hecho.	Benja	

ID y Título: US-21 Agregar Clase

ID	Descripción	Responsable	Est. Hrs.
US-50	Handleo de errores en Backend.	Alison	
US-49	Documentar la API.	Alison	
US-60	Realizar test unitarios de los Controllers.	Lean	
US-65	(OPCIONAL)- Revisar cómo se eliminará a futuro la clase.	Lean	
US-58	Agregar la ruta de la page del formulario.	Matias	
US-47	Crear componente AddClass	Matias	
US-66	Comunicar notificaciones utilizando Toastify.	Matias	
US-48	Crear funcionalidad de consumo de Servicios.	Matias	
US-56	Crear el estilo de la page del formulario.	Matias	
US-59	Probar que la página funciona correctamente.	Benja	
US-68	Crear comando para levantar backend y frontend a la vez.		

2.3 D.O.D.: Definición de hecho:

Se considera que la historia de usuario está terminada cuando se cumplen las siguientes condiciones:
- La visualización cumple con lo esperado y respeta los criterios de aceptación definidos.
- El frontend y el backend están implementados y permiten realizar el flujo previsto para la funcionalidad.
- Se tienen hechas las pruebas mínimas de los casos exitosos y de algunos casos de error relevantes.

3. BITÁCORA DE REFINAMIENTO Y TÉCNICAS DE VERTICAL SLICING

Sprint	*Incidencia*	*Acción*	*Detalle y justificación del refinamiento*
#2	*Creación de historia de usuario: Clase*	*Se agrega US al product backlog.*	*La US de creación de clases requiere de la existencia de la clase. Lo que llevó a una división de la US aplicando vertical slicing.*
#2	*Creación de historia de usuario: Registro de*	*Se agrega US al product backlog.*	*No se especificaba qué ocurría con los cupos. Lo que llevó a una división de la US aplicando vertical slicing.*

	cupos		
#2	*Refinamiento de historia de usuario: Agregar clase*	*Se modifica US en Product Backlog.*	*Ambigüedades que requieren análisis y división en más US.*
#2	*Refinamiento de historia de usuario: Eliminar clase*	*Se modifica US en Product Backlog.*	*Un análisis de refinamiento llevó a su modificación y posterior creación de US.*

4. DAILY SCRUM:

10/10: Se convocó a una reunión con el PO, SM y el equipo de desarrollo para definir el Sprint planning.

- Las US elegidas fueron la 45 y 21, que tienen que ver con la creación de una clase y posterior visualización en el calendario.
- Se definió el objetivo del sprint.
- Sin impedimentos.

11/10: Reunión del SM con el equipo de desarrollo para definir las tareas de las US.

- Definimos las tareas necesarias para la realización de cada US.
- Cada desarrollador eligió sus tareas para el sprint.
- Sin impedimentos

12/10:

- El equipo de desarrolladores conversaron por Discord sobre inconsistencias en el sistema.
- Se plantea una solución, falta implementarlo.

13/10:

- Conversando por Discord con el equipo de desarrolladores, les surgieron dudas acerca del modelo de negocio.
- Se plantea una nueva reunión con el PO, SM y los devs, para evacuar dudas del modelo.

14/10:

- El equipo de desarrolladores reportan las subtareas hechas y quedan en revisión.
- Se hizo un hilo en Discord para llegar a un acuerdo de cambios en el modelo.

15/10:

- El equipo de frontend plantea un posible error en el backend.
- El error era una falsa alarma, el equipo de backend tenía todo bajo control.

16/10:

- El equipo de frontend estaba implementando una User Story y se dieron cuenta de un error.
- Se plantea una solución, falta implementarlo.

17/10:

- Se realizó la Sprint Review y la retrospectiva con todo el equipo.
- Se probaron los avances y se acordó dar por finalizado el sprint.
- Se identificaron aciertos y dificultades, y se asignaron los roles para el próximo sprint.
- Sin impedimentos relevantes.

5. SPRINT REVIEW (DOBLE INSTANCIA: GESTIÓN Y DEMO ACTIVA)

Durante la Sprint Review se decidió considerar terminada la historia de usuario, ya que se completaron las subtareas consideradas esenciales para cumplir con el objetivo del sprint donde cada una cumple con el DoD. Solo quedaron pendientes las tareas opcionales de una sola User Story.

La demostración permitió observar la creación de la clase, agregarla al calendario para luego visualizarla en el mismo y comprobar el avance realizado en la integración de los componentes necesarios. De esta manera, se pudo evaluar el resultado obtenido en relación con los criterios de aceptación definidos para la historia de usuario.

6. SPRINT RETROSPECTIVE

6.1 Recolección de datos:

Aciertos:
Haber implementado los acuerdos añadidos del sprint 1, hacer una división de tareas lo antes posible.

Simplicidad de las tareas iniciales: Las tareas planteadas fueron relativamente sencillas, lo que facilitó su comprensión y ejecución dentro del tiempo disponible.

Separación entre backend y frontend: Permitió distribuir las responsabilidades y avanzar en paralelo sobre las distintas partes de la aplicación.

Hacer hilos en Discord para debatir sobre soluciones que incumben al frontend y el backend.

Dificultades:
En un momento del sprint el desarrollo se frenó por ambigüedades que fueron surgiendo sobre el modelo de negocio.

Algunas actividades requerían que otras estuvieran terminadas previamente, lo que podía limitar el trabajo en paralelo y generar esperas durante el desarrollo.

En las reuniones se dificulta optimizar el tiempo.

6.2 Indagar causas:

No se llegó a comprender del todo el modelo del negocio, por lo tanto surgen ambigüedades de cómo implementar las User Story.

Otra de las causas es que no darle prioridad a las subtareas hizo que el retraso de un equipo perjudique el desarrollo del otro.

6.3 Decidir qué hacer:

Para mejorar la organización y el desarrollo de los próximos sprints, se proponen las siguientes acciones:

Optimizar las reuniones, definiendo de antemano un tiempo de reunión con los temas y si no alcanza, pasar a otro canal de comunicación.

Analizar qué tareas requieren otras actividades previas y asignarles prioridad dependiendo si otra integrante del equipo la necesita para su desarrollo.

Por último, para el problema de las ambigüedades del modelo de negocio, dudas en general, decisiones de implementación, vamos a crear un nuevo canal de comunicación (parecido a una canal de notas), donde se documentaran las decisiones importantes y las dudas que surjan. Con esto, vamos a evitar que se hable en distintos canales de comunicación sobre el mismo tema.